

Delta-Tabellen anlegen: CTAS-Optionen, Constraints und Cloning

Section 1 · Databricks Intelligence Plattform · Quelle: Udemy-Kursmaterial „Set Up Delta Tables“, ergänzt mit Databricks-Dokumentation

Section 2, Kapitel 1 dieser Dokumentation hat CTAS (`CREATE TABLE AS SELECT`) bereits im Ingestion-Kontext eingeführt – als Methode, um Rohdaten aus Cloud-Speicher in eine Bronze-Tabelle zu laden. Dieses Kapitel betrachtet CTAS aus einer anderen Perspektive: als allgemeines Werkzeug zum Anlegen von Delta-Tabellen, ergänzt um Table Constraints und das Klonen von Tabellen.

1. CTAS: Spalten filtern und umbenennen

Da eine CTAS-Anweisung im Kern eine `SELECT`-Abfrage ausführt, lassen sich dabei ganz normale Projektionen anwenden – einzelne Spalten auswählen, weglassen oder umbenennen:

```
CREATE TABLE table_1
AS SELECT col_1, col_3 AS new_col_3 FROM table_2;
```

2. Zusätzliche CTAS-Optionen

Die `CREATE TABLE ... AS`-Klausel lässt sich um mehrere Optionen erweitern, die auch bei einer klassischen `CREATE TABLE`-Anweisung verfügbar sind:

```
CREATE TABLE new_table
COMMENT "Contains PII"
PARTITIONED BY (city, birth_date)
LOCATION '/some/path'
AS SELECT id, name, email, birth_date, city FROM users;
```

- `COMMENT` – ein beschreibender Kommentar, der die Tabelle im Katalog leichter auffindbar macht (hier z. B. ein Hinweis auf enthaltene personenbezogene Daten).
- `PARTITIONED BY` – physische Aufteilung der Daten in Unterordner nach einer oder mehreren Spalten (Details und Fallstricke dazu in Section 6, Kapitel 2).

- `LOCATION` – macht aus der neuen Tabelle eine External Table an einem frei gewählten Speicherort (siehe voriges Kapitel dieser Section).

Anstelle von `PARTITIONED BY` lässt sich bei der Tabellenerstellung ebenso die modernere `CLUSTER BY`-Klausel angeben – syntaktisch an derselben Stelle, aber ohne die Nachteile klassischer Partitionierung (siehe Section 6, Kapitel 3):

```
-- Explizite Clustering-Spalten
CREATE TABLE new_table (id INT, city STRING, birth_date DATE)
CLUSTER BY (city, birth_date);

-- Databricks waehlt die Clustering-Spalten selbststaendig anhand des Abfrageverhaltens
CREATE TABLE new_table (id INT, city STRING, birth_date DATE)
CLUSTER BY AUTO;
```

`CLUSTER BY AUTO` setzt allerdings eine **Managed Table** mit aktivierter Predictive Optimization voraus (siehe voriges Kapitel dieser Section) – bei einer `LOCATION`-basierten External Table steht diese automatische Variante nicht zur Verfügung. `PARTITIONED BY`, `CLUSTER BY` und `ZORDER` (über `OPTIMIZE`) schließen sich dabei gegenseitig aus: Für eine Tabelle gilt zu jedem Zeitpunkt nur eines dieser drei Datenlayout-Verfahren.

3. CREATE TABLE vs. CTAS im Vergleich

	CREATE TABLE (klassisch)	CTAS
Schema	Manuelle Deklaration erforderlich	Wird automatisch aus dem Abfrageergebnis abgeleitet – keine manuelle Deklaration möglich
Zustand nach Erstellung	Leere Tabelle	Sofort mit Daten befüllt
Daten laden	Erfordert einen separaten <code>INSERT INTO</code>	Erfolgt bereits während der Erstellung

```
-- Klassisch: manuelles Schema, danach leer
CREATE TABLE table_1 (col1 INT, col2 STRING, col3 DOUBLE);

-- CTAS: Schema und Daten kommen aus der Abfrage
CREATE TABLE table_1
AS SELECT col1, col2, col3 FROM table_2;
```

4. Table Constraints: NOT NULL und CHECK

Nach dem Anlegen – ob klassisch oder per CTAS – lassen sich einer Tabelle nachträglich Integritätsregeln hinzufügen. Databricks unterstützt dafür zwei Constraint-Typen: `NOT NULL` und `CHECK`. `CHECK`-Constraints funktionieren dabei wie eine gewöhnliche `WHERE`-Bedingung, die jeder neue Datensatz erfüllen muss:

```
ALTER TABLE orders
ADD CONSTRAINT valid_date CHECK (date > '2020-01-01');
```

Wichtige Voraussetzung: Bevor ein Constraint hinzugefügt werden kann, dürfen keine bereits vorhandenen Datensätze gegen die neue Regel verstoßen – Databricks prüft das beim Anlegen des Constraints. Ab diesem Zeitpunkt führt jeder Schreibversuch, der die Regel verletzt, zu einem Fehler und wird abgelehnt.

5. Informationelle Constraints: PRIMARY KEY und FOREIGN KEY

Zusätzlich zu `NOT NULL` und `CHECK` lassen sich unter Unity Catalog auch `PRIMARY KEY`- und `FOREIGN KEY`-Constraints deklarieren – mit einem entscheidenden Unterschied zu klassischen Datenbanken: Sie sind rein **informationell** und werden von Databricks **nicht erzwungen**.

```
-- Primaerschlüssel auf der Zieltabelle
ALTER TABLE customers ADD CONSTRAINT pk_customers PRIMARY KEY (customer_id);

-- Fremdschlüssel-Beziehung zwischen zwei Tabellen
ALTER TABLE orders ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_id) REFERENCES customers (customer_id);
```

Da Databricks die Eindeutigkeit bzw. referenzielle Integrität **nicht selbst prüft**, liegt es in der eigenen Verantwortung, vor dem Anlegen sicherzustellen, dass die Daten die deklarierte Beziehung tatsächlich einhalten – ein doppelter `customer_id`-Wert oder eine Bestellung ohne passenden Kunden führt zu keinem Fehler. Der Nutzen liegt stattdessen darin, dass Databricks diese Information für **Query-Optimierungen** heranziehen kann (z. B. um Joins effizienter zu planen), sowie darin, dass Modellierungs- und BI-Tools die Tabellenbeziehungen automatisch erkennen und visualisieren können. `FOREIGN KEY`-Constraints werden dabei nur für Unity-Catalog-Tabellen unterstützt, nicht für Tabellen im (legacy) Hive Metastore.

6. Nur das Schema kopieren: CREATE TABLE LIKE

Manchmal wird gar keine Datenkopie benötigt, sondern lediglich eine neue, leere Tabelle mit demselben Schema wie eine bestehende – etwa um eine Staging-Tabelle mit identischer Struktur anzulegen. Dafür eignet sich `CREATE TABLE ... LIKE` :

```
CREATE TABLE staging_orders LIKE orders;
```

Im Gegensatz zu `DEEP CLONE` / `SHALLOW CLONE` werden dabei zwar Schema, Partitionierung und Spalteneigenschaften übernommen, aber **keinerlei Daten und keine Verweise** auf die Quelldateien oder deren Transaktionslog – die neue Tabelle ist von Anfang an vollständig leer und eigenständig.

7. Delta-Tabellen klonen: Deep Clone vs. Shallow Clone

Um eine Kopie einer Tabelle zu erstellen – etwa um in einer Testumgebung gefahrlos mit Produktionsdaten zu experimentieren – bietet Delta Lake zwei Klon-Varianten:

```
-- Deep Clone: kopiert Daten UND Metadaten vollständig
CREATE TABLE table_clone
DEEP CLONE source_table;

-- Shallow Clone: kopiert nur die Transaktionslogs, keine Daten
CREATE TABLE table_clone
SHALLOW CLONE source_table;
```

Deep Clone	Shallow Clone
Kopiert Daten und Metadaten vollständig	Kopiert nur die Delta-Transaktionslogs, keine Datendateien
Kann bei großen Datenmengen einige Zeit dauern	Nahezu augenblicklich, da keine Daten bewegt werden
Erneutes Ausführen synchronisiert nachträgliche Änderungen der Quelle inkrementell	Ideal zum schnellen Testen von Änderungen, ohne die Originaltabelle zu berühren

In beiden Fällen gilt: Datenänderungen an der geklonten Tabelle wirken sich **nie** auf die Quelltable aus – Klone sind unabhängige Kopien, keine Referenzen.

Ergänzt aus der offiziellen Databricks-Dokumentation: Ein Shallow Clone verweist zunächst auf dieselben zugrunde liegenden Parquet-Dateien wie die Quelltable und lädt keine Daten neu –

deshalb ist zu beachten, dass ein `VACUUM` auf der *Quelltabelle* Dateien entfernen kann, auf die ein Shallow Clone noch verweist, was zu Lesefehlern im Klon führen kann. Deep Clones sind davon nicht betroffen, da sie eigenständige Kopien aller Datendateien besitzen.

Quelle: [Clone a table on Databricks – Databricks-Dokumentation](#)